

UPRAVLJANJE PROCESIMA

Proces

Najvažniji i jedni od najstariji koncepata operativnog sistema su *procesi* koji predstavljaju apstrakciju programa koji se izvršava. Oni podržavaju (pseudo)konkurentne operacije čak i kad je na raspolaganju samo jedan CPU kojeg pretvaraju u više virtualnih CPUa. Bez ovog tipa apstrakcije ne bi postojao današnji kompjuting.

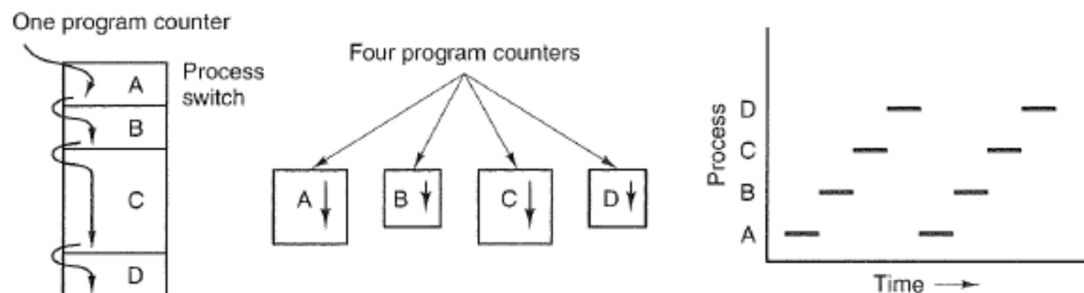
Rani računarski sistemi omogućavali su da se u određenom trenutku izvršava samo jedan program. On je imao potpunu kontrolu nad sistemom i pristup svim resursima. Nasuprot tome, današnji računarski sistemi obično rade nekoliko stvari istovremeno. Kao primjer se može uzeti jedan obični PC. Kad se pokrene sistem, pokreću se i mnogi procesi koji su često nepoznati ili čak sakriveni od samog korisnika. Jedan od takvih je proces koji čeka dolaznu elektronsku poštu. Drugi može u ime antivirusnog programa periodično provjeravati da li su na raspolaganju nove definicije za virusnu zaštitu. Pored toga, određeni procesi mogu se izvršavati, printati fajlove i prebacivati podatke na USB-drive dok sâm korisnik pretražuje Web. Svima njima se mora omogućiti da su istovremeno u memoriji, odnosno da se konkurentno (istovremeno) izvršavaju. U svakom slučaju, svim se ovim aktivnostima mora upravljati, a sistem multiprogramiranja, koji podržava višeprocesni rad, je adekvatno rješenje.

U svakom multiprogramskom sistemu CPU se vrlo brzo prebacuje sa jednog procesa na drugi, pri čemu svakog od njih izvršava redom veličine desetog ili stotog dijela milisekunde. Međutim, tačno je da se u datom trenutku vremena CPU izvršava samo jedan proces, ali u vremenskom intervalu od jedne sekunde, može raditi na nekoliko njih, te na taj način, stvarati iluziju paralelizma. Zato se u ovom kontekstu nekada govori o pseudoparalelizmu kako bi se napravila razlika u odnosu na stvarni hardverski paralelizam multiprocesorskih sistema koji imaju dva ili više CPU koji dijele istu fizičku memoriju. U općem slučaju, praćenje višestrukih, paralelnih aktivnosti je teško za čovjeka. Zato su projektanti operativnih sistema tokom godina razvili konceptualni model, poznat pod nazivom sekvencijalni (serijski) procesi, koji čini rad sa paralelizmom u računarskom sistemu jednostavnijim, ali zahtijeva čvršću kontrolu i modularnost programa.

U ovom modelu, svi softveri koji se izvršavaju na računaru, uključujući i operativni sistem, su organizovani u određeni broj sekvencijalnih procesa, ili jednostavno, procesa. Proces je samo primjer, odnosno slučaj u kojem se program izvršava, ili kraće rečeno, proces je program u izvršenju. Ujedno je i osnovna jedinica rada u modernim sistemima sa vremenskom raspodjelom (time-sharing). On uključuje trenutne vrijednosti programskog brojača, registara i varijabli. Konceptualno, svaki proces ima svoj vlastiti virtualni CPU. U stvarnosti, stvarni CPU se premješta naprijed-nazad sa jednog na drugi proces. Međutim, zbog lakšeg razumijevanja, jednostavnije je zamisliti skup procesa koji se izvršavaju (pseudo)paralelno, nego pratiti kako se CPU premješta sa programa na program. Ovo brzo premještanje CPUa naprijed-nazad se naziva *multiprogramiranje*.

Na slici 1.a, predstavljeno je multiprogramiranje četiri programa u memoriji. Četiri procesa (slika 1.b) sa vlastitom kontrolom izvršavaju se nazavisno jedan od drugog, to jest svaki od njih ima vlastiti logički programski brojač. Naravno da je samo jedan fizički programski brojač, tako da kad se proces izvršava, njegov se logički programski brojač učitava u stvarni programski brojač. Kad se proces završi, fizički programski brojač se spašava u procesovom logičkom programskom brojaču koji je pohranjen u memoriji. Na slici 1.c je predstavljen

dovoljno dug vremenski interval u kojem je vidljivo da su procesi napravili progres, ali u datom trenutku stvarno se izvršava samo jedan proces.



Slika1.: a) multiprogramiranje četiri programa. b) konceptualni model četiri nezavisna, sekvencijalna procesa. c) u datom trenutku je aktivan samo jedan.

U ovom primjeru pretpostavlja se da sistem ima samo jedan CPU. Međutim, današnji računari imaju CPU često sa dvije, četiri ili više jezgri na kojima se u datom trenutku izvršava po jedan proces.

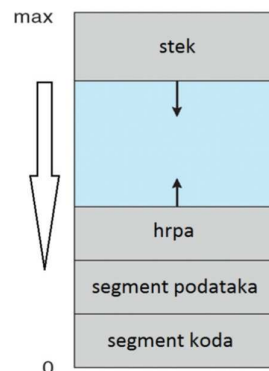
Razlika između procesa i programa nije toliko jasna, ali je njeno razumijevanje od presudnog značaja. U tu svrhu pomaže analogija. Razmotrimo ženu, domaćicu-majku, koja pravi rođendanski kolač svojoj kćerki. Ona ima recept za kolač i kuhinju dobro snabdjevenu svim potrebnim namirnicama (ulaznim jedinicama): brašno, jaja, šećer, ekstrakt vanilije itd. Po ovoj analogiji, recept je program, tj. algoritam izražen na prikladan način, žena je CPU, a sastojci kolača su ulazni podaci. Proces je aktivnost koja se sastoji od „slastičarke“ koja čita recept, dodaje sastojke i peče kolač. Sad zamislimo da ženi vrišteći dotrči sin kojeg je ubola pčela u ruku. Žena zabilježi u receptu dokle je došla (spašava se stanje tekućeg procesa), vadi pribor za prvu pomoć, i počinje pratiti uputstvo za dati slučaj uboda. Znači, procesor se prebacio sa jednog (pečenje kolača) na drugi proces (pružanje prve pomoći) koji je prioritetniji. Svaki proces ima različit program (recept kod jednog, a uputstvo za prvu pomoć kod drugog). Kad žena izvadi pčelinu žaoku iz sinove ruke, vraća se kolaču, nastavljajući od tačke gdje je stala. Ključna stvar je da je proces aktivnost neke vrste. Posjeduje program, ulaz, izlaz i stanje. Jedan procesor dijeli više procesa prema nekom algoritmu raspoređivanja kojim se određuje kada prestaje raditi na jednom procesu a počinje servisirati drugi.

Što je operativni sistem složeniji, to korisnici više od njega očekuju. Iako je njegov zadatak izvođenje korisničkih programa, on također treba voditi računa o različitim sistemskim zadacima koji ne ulaze u zadatke jezgre. Operativni sistem se dakle sastoji od skupa procesa: postoje procesi operativnog sistema koji izvršavaju sistemski kod, kao i korisnički procesi koji izvršavaju korisnički kod. Obje vrste procesa mogu da se izvršavaju konkurentno (istodobno, istovremeno), pri čemu se procesorsko vrijeme dijeli između njih. Dodjeljivanjem procesora različitim procesima operativni sistem može da poveća produktivnost računara.

Postavlja se pitanje šta su to aktivnosti procesora. Sistem sa paketnom obradom (batch) izvršava poslove (jobs), dok sistem sa vremenskom raspodjelom ima korisničke programe ili zadatke (task). Čak je i u sistemima sa jednim korisnikom kao što je Microsoft Windows, korisnik u stanju da izvršava nekoliko programa istovremeno (npr. program za obradu teksta, čitač Weba i klijent za e-poštu). I u slučaju kada korisnik može da pokrene samo jedan program, operativni sistem mora da podrži svoje interne aktivnosti, kao što je na primjer upravljanje memorijom. Po mnogo čemu sve te aktivnosti su slične, i zovemo ih *procesima*.

Izrazi *posao (job)* i *proces (process)* u terminologiji operativnih sistema su sinonimi i ravnopravno se koriste. Međutim, u posljednje vrijeme češće se koristi ovaj drugi, ali teorija i terminologija vezana za operativne sisteme razvijala se u vremenu kad je glavna aktivnost operativnih sistema bila procesiranje poslova. Zato ovdje napominjemo da se radi o dva ravnopravna izraza koji se odnose na isto.

Već je rečeno da je proces program u izvršenju, dok je **program** niz instrukcija koji ostvaruje neki algoritam, odnosno program je niz instrukcija koje se realizuju po unaprijed definisanoj proceduri. Međutim, proces je više od koda koji je poznat pod imenom **tekst segment ili segment kôda (text section)**. On sadrži i trenutnu aktivnost koju predstavljaju vrijednost **programskog brojača** i sadržaj registara procesora. Proces u općem slučaju sadržava još **stack¹ (stek, stog)** koji sadrži privremene podatke (parametri funkcija, povratne adrese i lokalne promjenjive), **segment podataka (data section)** koji sadrži globalne promjenjive. Proces može da sadrži i dinamičku memoriju ili **heap² (hrpa, gomila)**, tj. memoriju koja je dinamički alocirana tokom izvršavanja procesa (slika desno).



Dakle, za izvršavanje nekog procesa, mogu biti potrebna 4 dijela (vrste) memorije:

- **stack,**
- **heap,**
- **memorija za globalne promenljive, i**
- **memorija za kôd procesa.**

Naglasimo da program sam po sebi nije proces. Program je pasivan entitet, npr. fajl sa nizom instrukcija koji se čuva na disku (i često se zove **izvršni fajl** ili executable file), dok je proces aktivan entitet, sa programskim brojačem koji ukazuje na sljedeću instrukciju koja treba da se izvrši i sa pridruženim skupom resursa. *Program postaje proces kada se njegov izvršni fajl učitava u memoriju.* Dvije najčešće tehnike koje se koriste za učitavanje izvršnog fajla su: dupli klik (kad je izvršni fajl predstavljen ikonom) i unošenje imena izvršnog fajla u komandnu liniju (npr. prog.exe).

Iako sa istim programom mogu biti povezana dva procesa, ti procesi se posmatraju kao nezavisni skupovi instrukcija za izvršavanje. Na primjer, nekoliko korisnika može da pokrene različitu kopiju istog programa za e-poštu, ili isti korisnik može da pokrene nekoliko kopija Web browser-a. Svi oni su posebni procesi, pa iako im je segment kôda (odnosno tekst segment) isti, podaci, hrpa i stek im se razlikuju. Često se sreće i proces koji tokom izvršavanja obuhvata druge procese, ali o tome nešto kasnije.

¹ **Stek** je apstraktni tip podataka koji služi za pohranu niza istovrsnih elemenata. Specifičan je po ograničenom pristupu svojim elementima, tačnije, obično omogućava upis i ispis po principu "zadnji koji ulazi - prvi izlazi" (engl. LIFO - last in, first out, ili FILO – first in, last out; za razliku od mehanizma FIFO – first in, first out, ili LILO – last in, last out, koji se veoma rijetko koristi. Npr., imate polje brojeva koje želite sortirati. Nađite u polju najveći i gurnite ga na stek (i maknite iz polja); ponavljajte ovo tako dugo dok ima brojeva u polju. Kada polje ostane prazno, povadite onoliko elemenata sa steka koliko je elemenata bilo polju i vraćajte ih u polje od prve pozicije na dalje. Što ste dobili? Sortirano polje! Stek je vrlo koristan kod rekurzija, a pogotovo kada rekurzivne funkcije pretvarate u nerekurzivne. Tada se obavezno koristi stekom, ali o tome na nekim drugim predmetima.

² **Heap** je dio memorije koji je rezervisan za dinamičke promjenjive

Stanja procesa

Tokom izvršavanja, proces mijenja stanja. Stanje procesa djelimično je određeno njegovim trenutnim aktivnostima. Svaki proces može da bude u nekom od sljedećih stanja:

- **New**: proces se kreira.
- **Running**: instrukcije se izvršavaju.
- **Waiting**: proces čeka da se desi neki događaj (npr. završetak U/I operacije ili dolazak signala).
- **Ready**: proces čeka da bude dodijeljen procesoru.
- **Terminated**: izvršavanje procesa je završeno.

Nazivi stanja su proizvoljni i mijenjaju se od sistema do sistema, ali se stanja koja predstavljaju susreću u svim operativnim sistemima. Postoje i operativni sistemi u kojima su stanja procesa još detaljnije razrađena. *Važno je razumjeti da u bilo kojem trenutku jedan procesor može da izvršava samo jedan proces* (tj. samo jedan od svih procesa može da bude u stanju *Running*), dok postoje mnogi procesi koji mogu biti u stanju *Ready* i *Waiting*. Dijagram stanja procesa prikazan je na donjoj slici.



Dijagram stanja procesa

Stanje *New* odgovara procesu koji je upravo definisan. To znači da je operativni sistem obavio sve neophodne operacije za njegovo kreiranje (npr. dodijelio mu je identifikator), ali još nije počeo da ga izvršava, tj. proces još uvijek nije u radnoj memoriji već se još uvijek nalazi u sekundarnoj memoriji (obično na disku).

Proces se nalazi u fazi *Terminated* kada se izvrši njegova posljednja instrukcija, kada se izvršavanje prekine zbog fatalne greške ili kada neki drugi proces sa odgovarajućim ovlaštenjem zatraži da se on završi. Proces u stanju *Terminated* više nije kandidat za izvršavanje; operativni sistem još izvjesno vrijeme čuva podatke povezane sa njim da bi pomoćnim programima omogućio da dobiju neophodne informacije, a zatim briše sve podatke o njemu iz sistema.

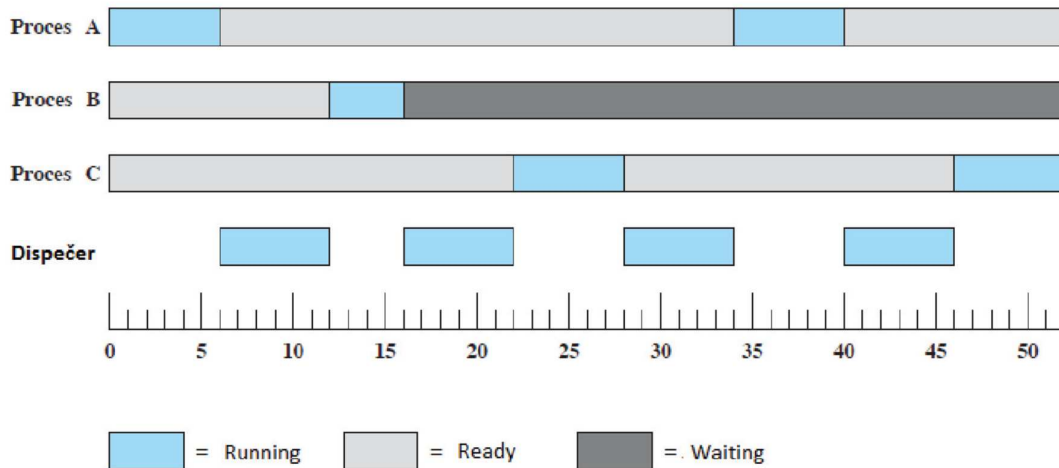
Dijagram stanja procesa na gornjoj slici pokazuje da su mogući sljedeći prelazi između stanja:

- *New* - *Ready*: Proces je spreman da počne sa izvršavanjem na procesoru. U većini sistema postoji ograničenje broja procesa koji se izvršavaju, odnosno količine memorije dodijeljene procesima, da prevelik broj procesa ne bi ugrozio performanse sistema.
- *Ready* - *Running*: Kada dođe trenutak za izbor novog procesa koji će se izvršavati, operativni sistem (odnosno raspoređivač) će izabrati neki od procesa u stanju *Ready*.
- *Running* - *Terminated*: Proces koji se trenutno izvršava okončava se ako je izvršena njegova posljednja instrukcija ili ako iz nekog drugog razloga njegovo izvršavanje treba

da se završi.

- *Running - Ready*: Najčešći razlog za ovaj prelaz je to što je proces koji se trenutno izvršava potrošio svoje vrijeme koje mu je dodijeljeno za izvršavanje; gotovo svi operativni sistemi sa vremenskom raspodjelom podržavaju ovakav pristup. Postoje i sistemi u kojima je primijenjen prioritet procesa, pa se prelaz između ova dva stanja dešava ako se za izvršavanje prijavi proces sa višim prioritetom.
- *Running - Waiting*: Proces se šalje u stanje *Waiting* ako zahtijeva nešto na šta mora da sačeka, npr. uslugu koju operativni sistem ne može trenutno da mu pruži, neki resurs ili U/I operaciju koja mora da se obavi da bi se izvršavanje nastavilo.
- *Waiting - Ready*: Ovaj prelaz se dešava kada se desi događaj na koji je proces čekao.

Donja slika prikazuje jedan primjer prelazaka tri procesa između različitih stanja tokom izvršavanja.



Primjer prelaska procesa između različitih stanja tokom izvršavanja

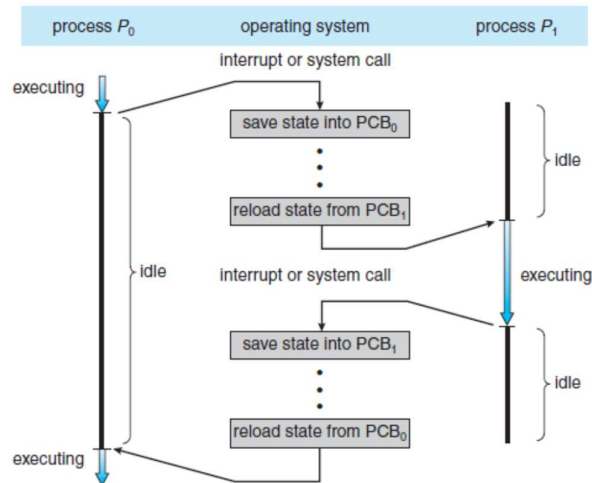
Blok za kontrolu procesa

Svaki proces je u operativnom sistemu predstavljen blokom za kontrolu procesa (PCB, Process Control Block), koji se još u literaturi naziva i blok za kontrolu zadataka (Task Control Block – TCB), slika desno. On sadrži mnoge informacije koje se tiču određenog procesa, a uključuje sljedeće:

- **Identifikator procesa** (process ID, PID): Jedinstveni identifikator koji se pridružuje svakom procesu da bi se razlikovao od ostalih.
- **Stanje procesa** (stanje može da bude *New*, *Ready*, *Running*, *Waiting* itd.).
- **Programski brojač** (brojač sadrži adresu sljedeće instrukcije koju proces treba izvršiti).
- **Sadržaj registara procesora** (Registri se razlikuju po broju i tipu, zavisno od računarske arhitekture. Uključuju: akumulatore, registre indeksa, stek pokazivače, i registre opće namjene, kao i informacije koje se tiču stanja određenog koda. Tokom izvršavanja procesa može doći do prekida. U slučaju prekida, ove informacije, zajedno sa programskim brojačem, moraju biti spašene (sačuvane) da bi se izvršavanje procesa kasnije pravilno nastavilo) (slika dole).
- **Informacije o raspoređivanju** (ove informacije uključuju prioritet procesa, pokazivače na redove za raspoređivanje kao i sve druge parametre bitne za raspoređivanje procesa).

Identifikator procesa (PID)
Stanje
Prioritet
Programski brojač
Registri
Podaci o memoriji
Podaci o otvorenim datotekama
...

- **Informacije o upravljanju memorijom** (vrijednost osnovnih i graničnih registara, steka, sadržaj tabela bitnih za upravljanje memorijom).
- **Informacije za praćenje** (utrošeno procesorsko i ukupno vrijeme, vremenska ograničenja, brojevi korisničkih naloga, brojevi procesa i sl.).
- **Informacije o U/I statusu** (spisak U/I uređaja dodijeljenih procesu, spisak otvorenih fajlova itd.).



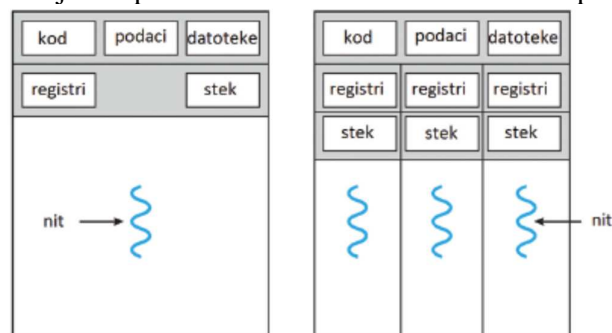
Blok za kontrolu procesa (PCB) je najvažnija struktura podataka u operativnom sistemu. Svaki PCB sadrži sve informacije o procesu koje su potrebne operativnom sistemu. Ove blokove čitaju i mijenjaju gotovo svi moduli operativnog sistema, uključujući one za raspoređivanje, alokaciju resursa, obradu prekida itd.

Niti

Model operativnog sistema o kojem do sada bilo riječi podrazumijeva da svaki proces ima jedinstven adresni prostor i tok kontrole (tzv. nit), što u stvari i jeste definicija procesa. Međutim, česte su situacije u kojima je poželjno imati višestruke tokove kontrole u istom adresnom prostoru koje se izvršavaju kvaziparalelno, kao da su (skoro) odvojeni procesi koji dijele adresni prostor. Na primjer, kad proces izvršava program za obradu teksta, izvršava se jedna nit instrukcija. Ova jedna nit kontrole dopušta procesu izvođenje samo jednog zadatka u datom trenutku. Korisnik ne može istovremeno kucati i pokretati provjeru pravopisa i gramatike. Većina današnjih operativnih sistema podržavaju prošireni koncept procesa koji podrazumijeva izvršavanje istog procesa u više niti (multithreading). Drugim riječima to znači da proces može izvoditi nekoliko zadataka istovremeno.

Prije nego što je uveden pojam niti, za istu svrhu primjenjivana je tehnika kreiranja procesa-djece. Međutim, kreiranje procesa zahtijeva dosta vremena i resursa. Rad sa nitima je neuporedivo efikasniji. Zato se za niti kaže da su „**laki procesi**“ (lightweight processes), za razliku od standardnih procesa koji se nazivaju „teški“ procesi (heavyweight processes). Nit je mehanizam kojim se obezbjeđuje nezavisno izvršavanje jednog zadatka unutar procesa. Ovo svojstvo naročito dolazi do izražaja kod višeprocorskih i višejezgrenih sistema, kod kojih se više niti može izvršavati istovremeno, dok se u jedno-procorskom sistemu može stvoriti privid njihovog istovremenog izvršavanja.

Nit je osnovna jedinica korištenja CPU-a. Višenitni proces sadrži nekoliko tokova kontrole tj. niti unutar istog adresnog prostora. To znači da sve niti istog procesa međusobno dijele njegove resurse i segment koda, nalaze se u istom adresnom prostoru i pristupaju istim podacima. Međutim, svaka nit unutar procesa ima



sopstveni identifikator, stanje (*Ready, Running, Waiting* itd.), programski brojač, skup registara i stek. Dakle, kod ovih sistema PCB je proširen tako da sadrži i informacije za svaku nit. Gornja slika ilustruje razliku između standardnog procesa koji se izvršava u jednoj niti i višenitnog procesa.

Pitanje je zašto imati drugu vrstu procesa unutar samog procesa? Nekoliko je prednosti za postojanje ovih miniprocasa koji se nazivaju niti. Mogu se podijeliti u četiri kategorije:

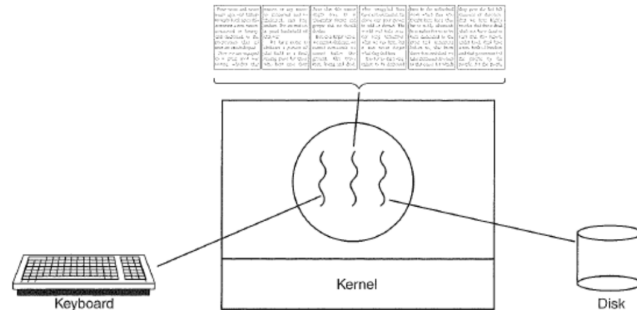
1. **Odziv.** Višenitnost dopušta programu izvođenje čak i u slučaju kad je jedan njegov dio blokiran ili kad izvršava dugu operaciju. Na taj način se povećava odziv prema korisniku. Ova je karakteristika korisna kod dizajniranja korisničkih interfejsa. Npr., šta bi se desilo kad korisnik klikne na dugme koje rezultira izvođenjem dugotrajne operacije? Kod jednonitnih sistema ne bi bilo odziva prema korisniku dok se ova operacija ne završi. S druge strane, ako bi se ovakva operacija izvodila kao zasebna nit, onda bi sistem mogao odgovoriti korisniku.
2. **Dijeljenje resursa.** Procesi mogu dijeliti resurse samo preko tehnika dijeljenja memorije ili prosljeđivanja poruka. Ove tehnike definiše programer. Međutim, prema svojoj definici niti dijele memoriju i resurse procesa kojem pripadaju. Dijeljenjem koda i podataka moguće je imati nekoliko različitih niti kontrole unutar istog adresnog prostora.
3. **Ekonomičnost.** Alociranje memorije i resursa procesu košta. Pošto niti dijele resurse procesa kojem pripadaju, ekonomičnije je kreiranje niti sa izmjenom konteksta. U općem slučaju više je vremena potrebno za kreiranje i upravljanje procesima nego nitima. Razlog je što su niti laki procesi, dakle, lakše se kreiraju i uništavaju od procesa. Kod mnogih sistema kreiranje niti je 10-100 puta brže od kreiranja procesa. Dobro je imati ovo svojstvo kada brojne niti trebaju dinamične i brze promjene.
4. **Skalabilnost.** Koristi od višenitosti mogu biti čak i veće kod višeprocorske arhitekture kod koje se niti mogu izvoditi paralelno na različitim procesorskim jezgrama. Jednonitni procesi mogu se izvoditi samo na jednom procesoru bez obzira koliko ih je raspoloživo.

Pored OS-a, i novi aplikativni programi su višenitni. Aplikacija se obično implementira kao odvojen proces sa nekoliko kontrolnih niti. Postojanje niti najbolje je sagledati na konkretnom primjeru programa za obradu teksta. Ovi programi obično na ekranu prikazuju dokument uređen tačno onako kako će izgledati nakon štampanja. Neka korisnik piše knjigu. Sa njegove tačke gledišta, najlakše je čitavu knjigu držati kao jedan dokument radi lakšeg pretraživanja, izmjene sadržaja i sl. Pored toga, svako poglavlje može biti jedinstven dokument. Međutim, ovo može biti prava smetnja kada se moraju učiniti globalne promjene u cijeloj knjizi jer se moraju urediti desetine dokumenata. Ako je knjiga sadržana u jednom dokumentu, obično jedna komanda može izvršiti takve vrste globalnih promjena (npr. promjena tipa slova).

Razmotrimo slučaj kada korisnik izbriše rečenicu sa prve od 800 stranica dokumenta. Nakon provjere tačnosti izmijenjene stranice, korisnik želi načini sljedeću promjenu na 600. stranici, te odgovarajućom komandom kaže programu da ode na tu stranicu, tražeći na primjer frazu koja se nalazi samo na toj stranici. Program je na taj način natjeran da na licu mjesta preuredi sve do 600. stranice jer ne zna koja je njena prva linija dok ne obradi sve prethodne stranice. Zato se može pojaviti znatno kašnjenje prije nego što se prikaže 600. stranica, pa je korisnik nezadovoljan.

Ovdje mogu pomoći niti. Neka je program napisan kao program sa dvije niti. Jedna je u interakciji sa korisnikom, a druga radi preuređivanje u pozadini. Čim rečenica sa prve stranice se izbriše, interaktivna nit kaže niti za preuređivanje da preuredi cijelu knjigu. U međuvremenu interaktivna nit osluškuje tastaturu i miš i reaguje na jednostavne komande kao što je pomicanje (scrolling) prve stranice, dok druga nit u pozadini „luduje“ od posla. Uz malo sreće, preuređivanje će se dovršiti prije korisnikovog zahtjeva da vidi 600. stranicu, pa može biti odmah prikazana.

Uzimajući u obzir gornji primjer mo-guće se zapitati zašto ne dodati i treću nit? Mnogi programi za obradu teksta imaju svojstvo automatskog spašavanja cijelog fajla na disk svakih nekoliko minuta kako bi se korisnik zaštitio od gubitka podataka u slučaju pada napona, aplikacije ili sistema. Treća nit može se baviti sigurnosnom kopijom dokumenta na disku bez ometanja druge dvije. Gore opisani model sa tri niti predstavljen je na gornjoj slici.



Da program za obradu teksta ima samo jednu nit, onda, kad god započne pravljenje sigurnosne kopije, komande sa tastature i miša bi bile ignorisane dok se prethodni posao ne završi, pa je korisnik opet nezadovoljan zbog slabih performansi. Rješenje ove situacije je da komande sa tastature i miša preko prekida utiču na aktivnost izrade sigurnosne kopije. Na taj način bi se dobile dobre performanse, ali ovakav programski model projektovanja prekida je veoma složen. Sa tri niti, programski model je mnogo jednostavniji. Prva nit samo vrši interakciju sa korisnikom, druga preuređuje dokument, a treća periodično upisuje sadržaje sa RAM-a na disk. (pokušajte zamisliti sličnu situaciju u EXCElu).

Kod ovog primjera veoma je važno primijetiti da kada bi se koristila tri procesa, posao ne bi mogao biti izvršen jer sva tri moraju raditi na dokumentu. S druge strane, tri niti dijele memoriju i zato sve imaju pristup dokumentu koji se uređuje.

Drugi primjer, web pretraživač ima jednu nit za prikaz slika ili teksta, drugu za dobijanje podataka iz mreže. Ovakve aplikacije mogu znatno bolje iskoristiti mogućnosti višezgrednih (procesorskih) sistema jer istovremeno mogu izvoditi nekoliko paralelnih zadataka na više jezgri ili procesora.

Niti također imaju veoma važnu ulogu kod RPC (remote procedure call) sistema. RPC-ovi dopuštaju međuprocensnu komunikaciju obezbjeđujući komunikacijski mehanizam sličan običnoj funkciji ili pozivima procedura. Obično je RPC višenitni. Kada server primi poruku, onda odgovara na poruku korištenjem zasebne niti. Na ovaj način server uslužuje nekoliko konkurentnih zahtjeva.

Jezgre današnjih operativnih sistema su višenitne. Postoji nekoliko niti u jezgi, pri čemu svaka izvodi specifičan zadatak kao što je upravljanje uređajima, memorijom, ili prekidima. Npr., Linux koristi nit jezgre za upravljanje slobodnom memorijom u sistemu.

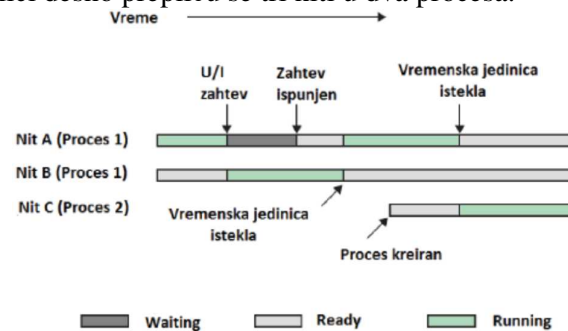
Prednosti korištenja niti nad kreiranjem novih procesa su višestruke:

- Nova nit se kreira i uništava mnogo brže nego novi proces, što čini programe interaktivnijim,

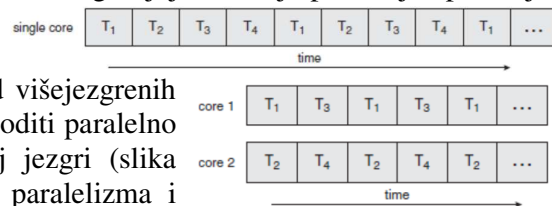
- Niti omogućuju efikasnije korištenje resursa,
- Prelazak između niti istog procesa je mnogo brži nego prelazak između različitih procesa,
- Niti omogućuju efikasniju komunikaciju unutar programa koji se izvršavaju, jer niti unutar istog procesa dijele memoriju i datoteke pa mogu da komuniciraju bez posredstva jezgre operativnog sistema.

Kao što je rečeno, u sistemu sa jednom procesorskom jezgrom multiprogramiranje omogućuje preplitanje niti u više procesa. U primjeru na slici desno prepliću se tri niti u dva procesa.

Izvršavanje prelazi sa jedne niti na drugu kada se nit koja se trenutno izvršava blokira ili istekne vremenska jedinica (kvantum vremena) dodijeljen procesoru. U ovom primjeru, nit C počinje da se izvršava kada istekne vremenski kvantum niti A, iako je u tom trenutku i nit B spremna za izvršavanje. Izbor između niti B i niti C je odluka raspoređivača, o kojima će biti riječi kasnije.



Dakle, kod jednojezgrenih sistema konkurentnost samo znači da se niti pri izvršavanju izmjenjuju na procesoru kao što je to predstavljeno na gornjoj slici koja prikazuje aplikaciju koja sadrži četiri niti (T_1 , T_2 , T_3 i T_4). Razlog je što jednojezgreni procesor u datom trenutku može izvršavati samo jednu nit. Međutim, kod višejezgrenih sistema konkurentnost znači da se niti mogu izvoditi paralelno jer sistem može dodijeliti zasebnu nit svakoj jezgri (slika desno). Ovdje treba napraviti razliku između paralelizma i konkurentnosti. Govorimo o paralelnom sistemu ako on može izvršavati više zadataka istovremeno. S druge strane, konkurentni sistem podržava da jedan zadatak dopušta ostalim da napreduju. Zato je moguće ostvariti konkurentnost bez paralelizma. Pošto niti procesa dijele isti adresni prostor i druge resurse, npr. otvorene datoteke, ako jedna nit izmjeni resurs, to će uticati na sve druge niti istog procesa. Zbog toga je neophodno sinhronizovati aktivnosti različitih niti da ne bi ometale jedna drugu ili narušavale strukturu podataka. Problemi sinhronizacije niti u suštini su isti kao problemi sinhronizacije procesa.



Korisničke i sistemske niti

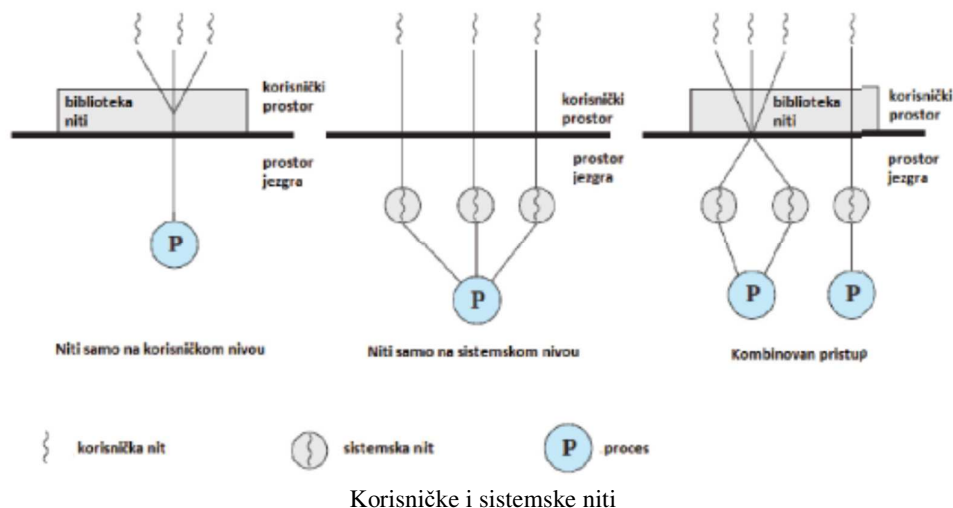
Operativni sistem može da podržava niti na nivou korisnika ili jezgre. Korisničkim nitima upravlja se bez podrške jezgre, dok nitima jezgre direktno upravlja operativni sistem. Skoro svi operativni sistemi podržavaju niti jezgre (kernel threads).

U slučaju korisničkih niti, posao upravljanja nitima obavlja aplikacija, a jezgro uopće ne zna za postojanje niti. Svaka aplikacija može da bude programirana tako da bude višenitna uz pomoć odgovarajuće biblioteke. **Biblioteka niti** (thread library) je interfejs (API) za kreiranje i uništavanje niti, razmjenu podataka i poruka između njih, raspoređivanje izvršavanja niti, čuvanje i rekonstruisanje konteksta niti.

Aplikacija standardno počinje izvršavanje u jednoj niti; aplikacija i njena nit dodijeljene su

jedinstvenom procesu kojim upravlja jezgro operativnog sistema. U svakom trenutku tokom izvršavanja aplikacija može da kreira novu nit koja će se izvršavati u istom procesu, što se postiže pomoću odgovarajuće funkcije u biblioteci niti. Biblioteka pravi novu strukturu podataka za nit, a zatim prepušta kontrolu nekoj od niti procesa koja se nalazi u stanju *Ready*, pridržavajući se utvrđenog algoritma raspoređivanja niti.

Jezgro operativnog sistema uopće ne zna za te aktivnosti, i nastavlja da radi sa višenitnim procesom na standardan način, raspoređujući ga u redove u zavisnosti od stanja. Donja slika ilustruje način rada korisničkih i sistemskih niti.



Slučaj (b) prikazuje „čiste“ niti jezgre. Jezgro održava informacije o kontekstu procesa u cjelini, kao i informacije o kontekstu pojedinačnih niti unutar procesa. Raspoređivanje unutar jezgre obavlja se na nivou niti, čime se prevazilaze dva važna ograničenja pristupa u slučaju (a) kada se koriste samo korisničke niti. Prvo, jezgro može istovremeno da raspoređuje više niti istog procesa na nekoliko procesora. Drugo, ako je jedna nit u procesu blokirana, jezgro može da pošalje drugu nit istog procesa na izvršavanje. Također, same rutine jezgre mogu da budu višenitne. S druge strane, korištenje niti jezgra zahtijeva prenos kontrole sa jedne niti na drugu unutar istog procesa, a to zahtijeva promjenu režima rada jezgre. Neki operativni sistemi koriste kombinovan pristup, prikazan, slučaj (c), koji primjenjuje i korisničke i niti jezgre. U takvom pristupu, niti se kreiraju u korisničkom prostoru, gdje se obavlja i najveći dio posla sinhronizacije i raspoređivanja niti u aplikaciji. Više korisničkih niti iste aplikacije mapiraju se na (manji ili jednak) broj niti jezgre. Programer može da podešava broj korisničkih niti za određenu aplikaciju i mašinu da bi postigao najbolje rezultate. Ovakav pristup je primijenjen u operativnom sistemu Solaris i omogućuje da se više niti unutar iste aplikacije paralelno izvršava na više procesora.

Dakle, prednosti korištenja korisničkih niti:

- Mijenjanje niti ne zahtijeva privilegije izvršavanja u sistemskom režimu jer se sve strukture podataka za upravljanje nitima nalaze u adresnom prostoru jednog procesa.
- Raspoređivanje se može implementirati za svaku aplikaciju posebno. U zavisnosti od toga o kojem programu se radi, u nekim slučajevima biće pogodnije kružno raspoređivanje, a u drugim princip prioriteta (govorićemo poslije o načinima raspoređivanja).
- Korisničke niti mogu da se izvršavaju na bilo kojem operativnom sistemu i ne zahtijevaju nikakve izmjene jezgre.

Ipak, korisničke niti imaju i dva velika nedostatka:

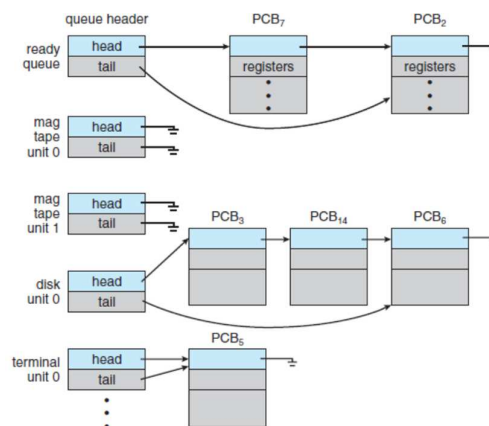
- Mnogi sistemski pozivi su blokirajući; ako korisnička nit obavi sistemski poziv, biće blokirane sve niti procesa.
- Višenitna aplikacija ne može da iskoristi prednosti multiprocesiranja. Jezgro dodjeljuje jednom procesu samo jedan procesor, što znači da se unutar procesa može izvršavati samo jedna nit.

Raspoređivanje procesa

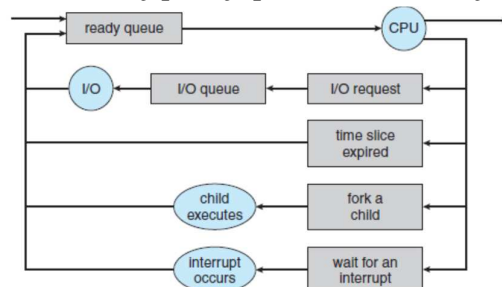
Cilj multiprogramiranja je da se u svakom trenutku izvršava neki proces, da bi se procesor maksimalno iskoristio. Cilj vremenske raspodjele je da procesor prelazi između procesa toliko često da korisnici mogu da komuniciraju sa svakim programom dok se izvršava. Da bi se ostvarili ti ciljevi, **raspoređivač procesa** (process scheduler) bira neki proces u stanju *Ready* (između svih procesa dostupnih za izvršavanje) i šalje ga procesoru na izvršavanje. U jednoprocorskom sistemu nikada se u određenom trenutku ne izvršava više od jednog procesa. Ako ima više procesa, svi ostali moraju čekati da se procesor oslobodi.

Redovi za raspoređivanje

Kako procesi ulaze u sistem, smještaju se u **red poslova** (job queue). Procesi, koji se nalaze u radnoj memoriji su spremni i čekaju na izvršenje, čuvaju se u listi koja se zove **red spremnih poslova** (ready queue). Ovaj red se u općem slučaju čuva kao ulančana (povezana) lista (linked list). Zaglavlje reda spremnih poslova sadrži pokazivače na prvi i posljednji PCB u listi. Svaki PCB sadrži polje za pokazivač na sljedeći PCB u redu spremnih poslova.



Sistem sadrži i druge redove. Kada se procesu dodijeli procesor, on se izvjesno vrijeme izvršava i na kraju završava izvršavanje, desi se prekid ili čeka na određeni događaj, npr. ispunjavanje U/I zahtjeva. Pretpostavimo da proces zahtjeva neki dijeljeni U/I uređaj, kao što je disk. Pošto u sistemu ima mnogo procesa, disk može da bude zauzet U/I zahtjevom nekog drugog procesa. U tom slučaju proces mora da čeka da se disk oslobodi. Spisak procesa koji čekaju na određeni U/I uređaj zove se **red uređaja** (device queue). Za svaki uređaj postoji poseban red uređaja (slika gore).



Raspoređivanje procesa obično se predstavlja dijagramom redova (slika desno). Svaki pravougaonik predstavlja red. Na slici su predstavljena dva tipa redova: red *Ready* i skup redova uređaja. Krugovi predstavljaju sredstva koja su na usluzi redovima, dok strelice ukazuju na tok procesa u sistemu.

Novi proces se u početku stavlja u red spremnih poslova gdje čeka dok ne bude izabran za izvršavanje, odnosno dok ne bude izabran za prosljeđivanje. Kada se proces dodijeli procesoru i započne izvršavanje, može da se desi neki od sljedećih događaja:

- proces može da zatraži U/I i u tom slučaju se smješta u U/I red,
- proces može da napravi novi potproces i da sačeka da se on završi,
- proces može prinudno da bude uklonjen iz procesora, zbog prekida, i da bude vraćen u red spremnih poslova.

U prva dva slučaja proces na kraju prelazi iz stanja *Waiting* u stanje *Ready* i vraća se u red spremnih poslova. Proces nastavlja taj ciklus dok se ne završi; tada se uklanja iz svih redova, a PCB i resursi koje je zauzimao se oslobađaju.

Raspoređivači

Proces se tokom svog životnog vijeka kreće kroz različite redove za raspoređivanje. Operativni sistem mora na neki način da bira procese iz tih redova. Biranje procesa obavlja se pomoću **raspoređivača** (scheduler).

Često se kreira više procesa nego što trenutno može da se izvršava. Ti procesi se smještaju u uređaj za čuvanje podataka (obično je to disk) radi kasnijeg izvršavanja. **Dugoročni raspoređivač** (long-term scheduler ili job scheduler) bira procese sa diska i učitava ih u memoriju. **Kratkoročni raspoređivač** (short-term scheduler) ili **CPU raspoređivač** (CPU scheduler) bira između procesa koji su u stanju *Ready* i nekom od njih dodjeljuje procesor.

Najvažnija razlika između ova dva raspoređivača jeste u frekvenciji izvršavanja. Kratkoročni raspoređivač mora često da bira novi proces za procesor. Proces se ponekad izvršava samo nekoliko milisekundi prije nego što biva prinuđen da sačeka na ispunjenje U/I zahtjeva. Često se kratkoročni raspoređivač pokreće na svakih sto milisekundi, i zbog toga mora da bude brz. Ako mu, na primjer, treba 10 milisekundi da odluči koji proces ide na izvršenje, i neka se on izvršava 100 ms, onda se $10/(100+10)=9\%$ CPU vremena troši na posao raspoređivanja.

Dugoročni raspoređivač se izvršava mnogo rjeđe; između kreiranja dva procesa može ponekad da prođe i nekoliko minuta. Dugoročni raspoređivač upravlja brojem procesa u memoriji tzv. stepen multiprogramiranja (degree of multiprogramming). Ako je on stabilan, onda prosječna brzina kreiranja procesa mora biti jednaka prosječnoj brzini procesa koji napuštaju sistem. Zato dugoročni raspoređivač treba da se poziva samo kada neki proces napusti sistem. Zbog dužeg intervala između izvršavanja, dugoročni raspoređivač ima na raspolaganju više vremena da odluči koji će proces izabrati za izvršavanje.

Veoma je važno da dugoročni raspoređivač pažljivo bira procese. Postoje procesi koji troše više vremena na U/I operacije (I/O-bound process), kao i procesi koji ih rjeđe generišu i većinu vremena provode u izračunavanjima (CPU bound). Važno je da dugoročni raspoređivač izabere dobru kombinaciju ove dvije vrste procesa. Ako su svi I/O bound procesi, *Ready* red će skoro uvijek biti prazan, pa će kratkoročni raspoređivač imati veoma malo posla. Međutim, ako su sve CPU bound procesi *Waiting* red uređaja će skoro uvijek biti prazan, pa će uređaji biti neiskorišteni, što znači da sistem kao i u prethodnom primjeru neće biti balansiran. Zato sistem sa najboljim performansama ima dobru kombinaciju I/O i CPU bound procesa.

Promjena konteksta (context switch)

Prekidi prouzrokuju prestanak rada procesora na trenutnom zadatku i prelazak u prekidnu rutinu jezgre. Takve operacije često se dešavaju u sistemima opće namjene. Kada se desi prekid, sistem mora da sačuva **kontekst** procesa koji se izvršava u procesoru da bi mogao da ga povrati kada se završi obrada prekida, što u suštini znači da procesor treba da suspenduje izvršavanje

tekućeg procesa i da ga kasnije nastavi. Kontekst je zapravo PCB procesa. On obuhvata vrijednosti procesorskih registara, stanje procesa i informacije o upravljanju memorijom. U općem slučaju, čuva se stanje trenutnog procesa, koji može biti u kernel ili korisničkom modu, a zatim se ono rekonstruiše da bi se nastavio rad.

Prebacivanje procesora na drugi proces zahtijeva izvođenje spašavanja stanja tekućeg procesa i stanja rekonstruiranja drugog procesa. Ovaj postupak se zove **promjena konteksta** (context switch). Kada se promijeni kontekst, jezgro operativnog sistema čuva kontekst starog procesa u njegovom PCBu i učitava kontekst novog procesa koji treba da se izvrši. Vrijeme utrošeno na promjenu konteksta je čist gubitak, jer sistem u tom periodu ne radi ništa korisno. Tipično vrijeme promjene konteksta je nekoliko ms, a zavisi od brzine memorije, broja registara čije vrijednosti se moraju kopirati i podržanih instrukcija, kao i od hardverske podrške.

Kreiranje procesa

Procesi se kod većine sistema mogu izvršavati konkurentno, pa se mogu kreirati i brisati dinamički. Zato sistemi moraju omogućiti mehanizam kreiranja i završavanja procesa.

Većina operativnih sistema procese razlikuje na osnovu cjelobrojnog identifikatora (PID). U općem slučaju, novom procesu će biti potrebni određeni resursi za izvršavanje zadatka. Kada operativni sistem iz bilo kojeg razloga odluči kreirati novi proces, postupiće po sljedećoj šemi:

- Novom procesoru dodijeliće jedinstveni identifikator (PID),
- Alociraće prostor u memoriji za nov proces,
- Inicijaliziraće PCB novog procesa,
- Staviće novi proces u odgovarajući red (npr. red procesa spremnih za izvršavanje),
- Napraviće ili ažurirati sve potrebne strukture podataka.

Postojeći proces može da kreira nekoliko novih procesa pomoću odgovarajućeg sistemskog poziva. Proces koji pravi nove procese zove se **proces roditelj**, a novi proces zove se potproces ili **proces dijete**. Postupak kreiranja novih procesa može se nastaviti tako da se kao rezultat dobije stablo procesa.

Kada proces roditelj napravi potproces, proces dijete će resurse moći dobiti direktno od operativnog sistema, ili će biti ograničen da koristi resurse koji su već dodijeljeni procesu roditelju. Ograničavanje procesa djeteta na korištenje resursa procesa roditelja onemogućava situacije u kojima bi neki proces mogao da preopteretiti sistem kreiranjem prevelikog broja potprocesa. Kada je riječ o izvršavanju procesa, postoje dvije mogućnosti:

1. proces roditelj može da nastavi da se izvršava konkurentno s procesima djecom, ili
2. može i da sačeka dok se neki od procesa djece ne završi.

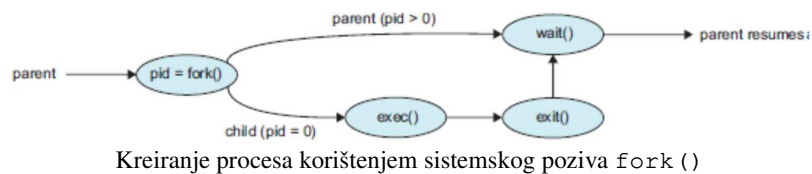
U pogledu adresnog prostora također postoje dvije mogućnosti:

1. segment koda procesa djeteta može da bude kopija procesa roditelja, ili
2. može i da izvršava neki drugi kod.

Kao primjer, proučimo kako se kreiraju procesi djeca u UNIX sistemima. Proces dijete se pravi pomoću sistemskog poziva `fork()`. Novi proces sadrži kopiju adresnog prostora procesa roditelja, što olakšava komunikaciju između procesa roditelja i njegovih potprocesa. Oba procesa (roditelj i dijete) nastavljaju da se izvršavaju od instrukcije iza sistemskog poziva `fork()`, uz jednu razliku: vrijednost koju poziv `fork()` vraća nakon svog izvršavanja procesu

djetetu je 0, dok se procesu roditelju vraća vrijednost jednaka PID-u procesa djeteta, koji je cijeli broj.

Obično, nakon sistemskog poziva `fork()`, jedan od dva procesa koristi sistemski poziv `exec()` da bi se zamijenio memorijski prostor dodijeljen procesu novim programom. Sistemski poziv `exec()` učitava binarni fajl u memoriju (pri tom uništava sliku programa koji sadržava `exec()` sistemski poziv) i počinje da je izvršava. Na taj način dva procesa su u stanju da u početku komuniciraju, a zatim pođu svaki svojim putem. Proces roditelj nakon toga može da kreira nove procese djecu, a ako nema šta da radi dok se proces djeteta izvršava, može da iskoristi sistemski poziv `wait()` da bi izašao iz reda spremnih poslova dok se izvršavanje procesa djeteta ne završi. Kad se proces djeteta izvrši (implicitno ili eksplicitno pozivajući sistemski poziv `exit()`), proces roditelj se nastavlja gdje je stao nakon sistemskog poziva `wait()`, te se završava koristeći sistemski poziv `exit()`. Donja slika.



Završavanje procesa

Proces se završava kada se izvrši njegova posljednja naredba, nakon čega on zahtijeva od operativnog sistema da ga obriše sistemskim pozivom `exit()`. Tada proces obično vraća procesu roditelju neku cjelobrojnu vrijednost pomoću sistemskog poziva `wait()`. Operativni sistem oslobađa sve resurse koje je proces koristio (memoriju, fajlove, U/I bafere i sl.)

Završavanje može da se desi i u drugim okolnostima. Proces može da zahtijeva da se neki drugi proces okonča pomoću odgovarajućeg sistemskog poziva. Takav poziv obično može da uputi samo roditelj procesa koji treba da se završi, jer bi u suprotnom korisnici mogli namjerno da prekidaju procese drugih korisnika sistema. Proces roditelj mora da zna koji su procesi njegova djeca, odnosno mora znati njihov identitet. To je i razlog zbog koga sistemski poziv za kreiranje procesa djeteta prosljeđuje njegov PID procesu roditelju koji ga je kreirao.

Roditelj može završiti izvršavanje svoje djece iz mnogo različitih razloga, kao što su:

- dijete koristi resurse više nego mu je dodijeljeno (u tom slučaju roditelj mora imati mehanizam provjere stanja djeteta kako bi provjerio ovaj slučaj),
- zadatak koji je dodijeljen djetetu se više ne zahtijeva,
- roditelj se završava, a operativni sistem ne dozvoljava djetetu nastavak nakon završetka roditelja, itd.

Literatura:

1. Avi Silberschatz, Peter Baer, Galvin Greg Gagne, *Operating System Concepts 9th Edition*, 2013. godine, John Wiley & Sons, Inc., 111 River Street, Hoboken, New Jersey
2. Andrew S. Tanenbaum, *Modern Operating Systems 3rd Edition*, Pearson Education Inc., 2008.